



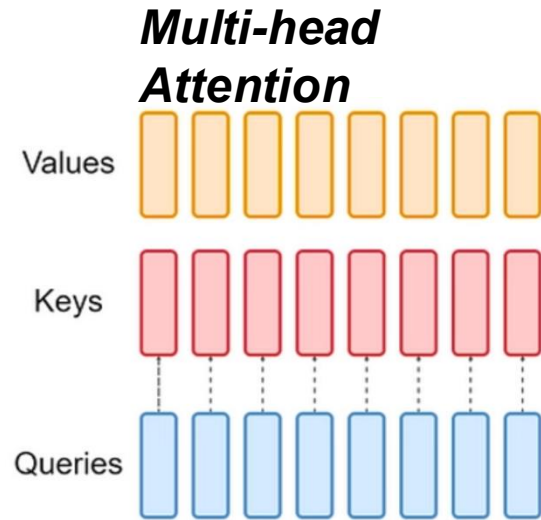
CS 498: Distributed Training Overview

Fan Lai

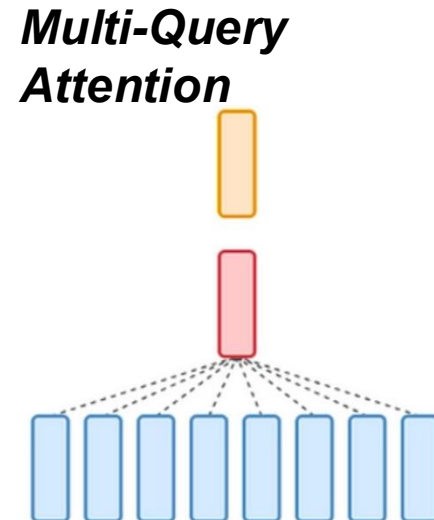
The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

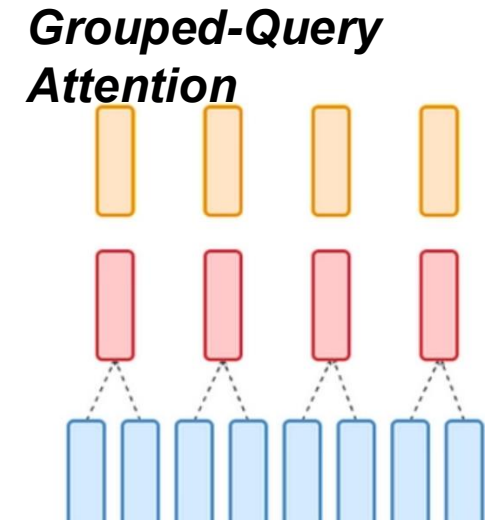
Recall: Multi-Head/Query Attention



Each head digests QKV separately

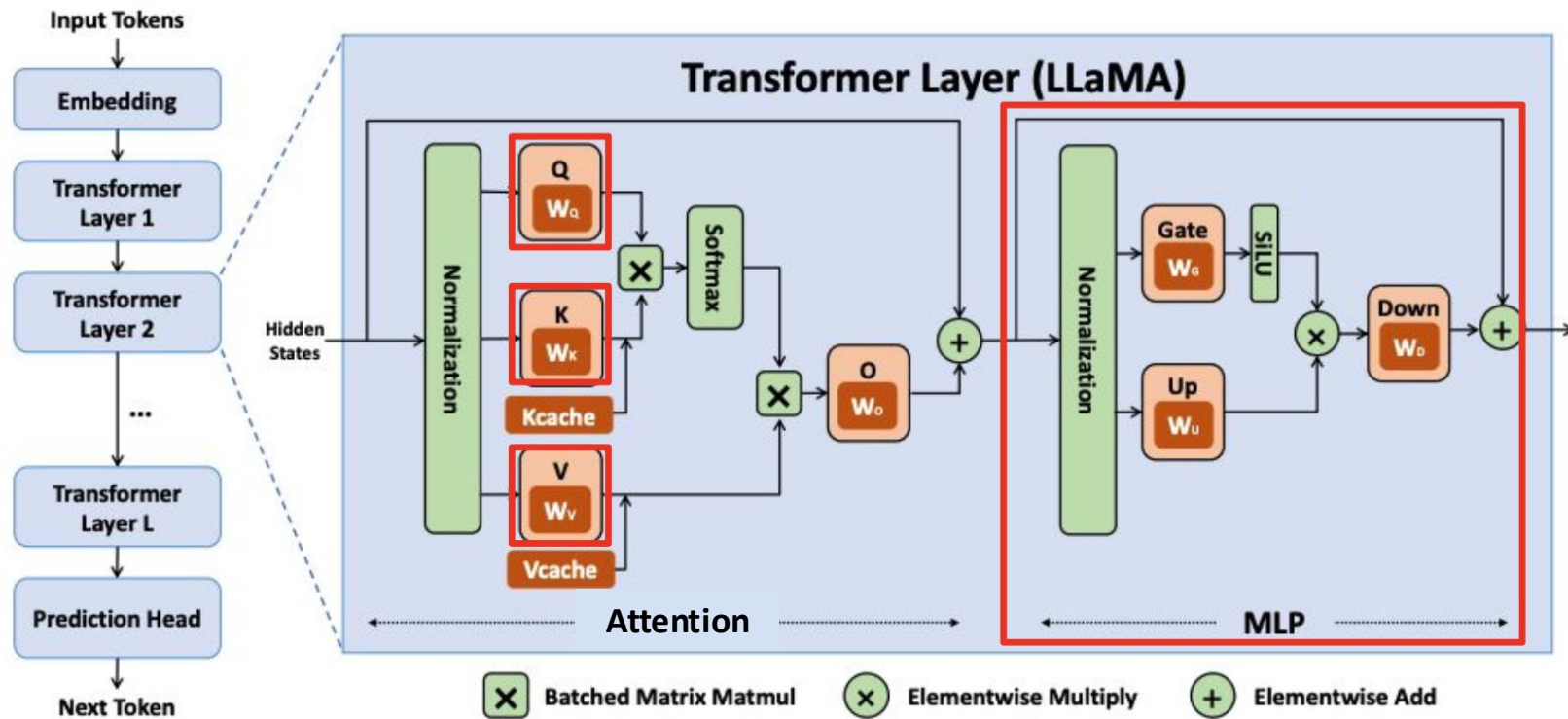


Share single key and value heads across query heads



*Share single key and value heads for **each group** of query heads*

Recall: Multi-Head/Query Attention



Recall: Multi-Head/Query Attention



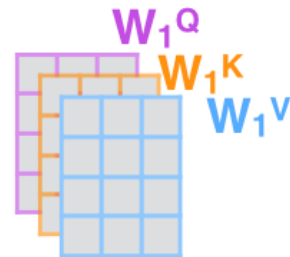
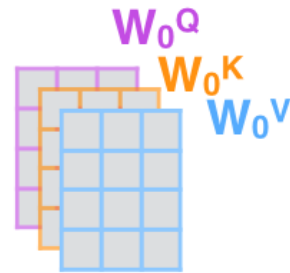
1) This is our
input sentence*

Thinking
Machines

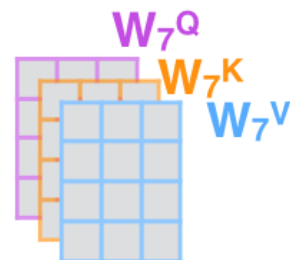
2) We embed
each word*



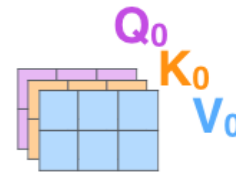
3) Split into 8 heads.
We multiply X
with weight matrices



...



4) Calculate attention
using the resulting
 $Q/K/V$ matrices



...



Recall: Multi-Head/Query Attention



1) This is our input sentence*

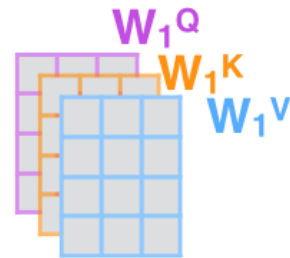
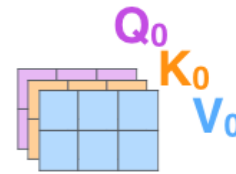
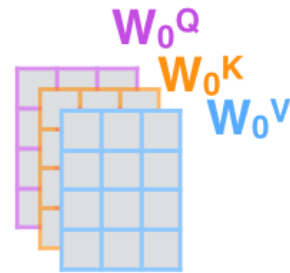
2) We embed each word*

3) Split into 8 heads.
We multiply X with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

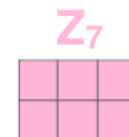
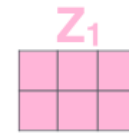
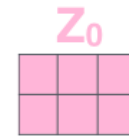
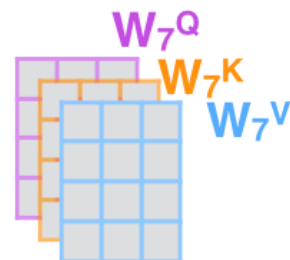
Thinking
Machines



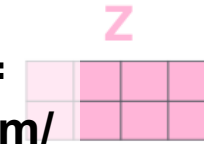
...

...

...



#_heads =
hidden_dim/
size_per_head



MLP
Layer

- **Distributed Training**
 - Problem
 - Challenge
 - History
 - Parameter Server based Data Parallelism (DP)
 - Communication terminologies
- **First panel discussion**

- Given model f , data set $\{x_i, y_i\}_{i=1}^N$
- Minimize the loss between predicted labels and true labels:

$$\text{Min } \frac{1}{N} \sum_{i=1}^N \text{loss}(f(x_i, y_i))$$

- Common loss function
 - Cross-entropy, MSE (mean squared error)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

- Given model f , data set $\{x_i, y_i\}_{i=1}^N$
- Minimize the loss between predicted labels and true labels:
$$\text{Min } \frac{1}{N} \sum_{i=1}^N \text{loss}(f(x_i, y_i))$$
- Common loss function
 - Cross-entropy, MSE (mean squared error)
- Common way to solve the minimization problem
 - Adaptive learning rates optimizers (e.g., Adam)
 - Stochastic gradient descent (SGD)

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

For $t = 1$ to T (iters)

Backward pass Forward pass

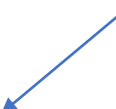
$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^N \nabla \left(\text{loss}(f_w(x_i, y_i)) \right)$ // compute derivative and update

$w -= \Delta w$ // apply update

End

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

For $t = 1$ to T (iters)


$$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^N \nabla \left(\text{loss}(f_w(x_i, y_i)) \right) \quad // \text{ compute derivative and update}$$

$w \leftarrow w + \Delta w$ // apply update

End

Can we accelerate it?

- Model f_w is parameterized by weight w
- $\eta > 0$ is the learning rate

Increase the batch size increases throughput but eventually bounded by single GPU compute

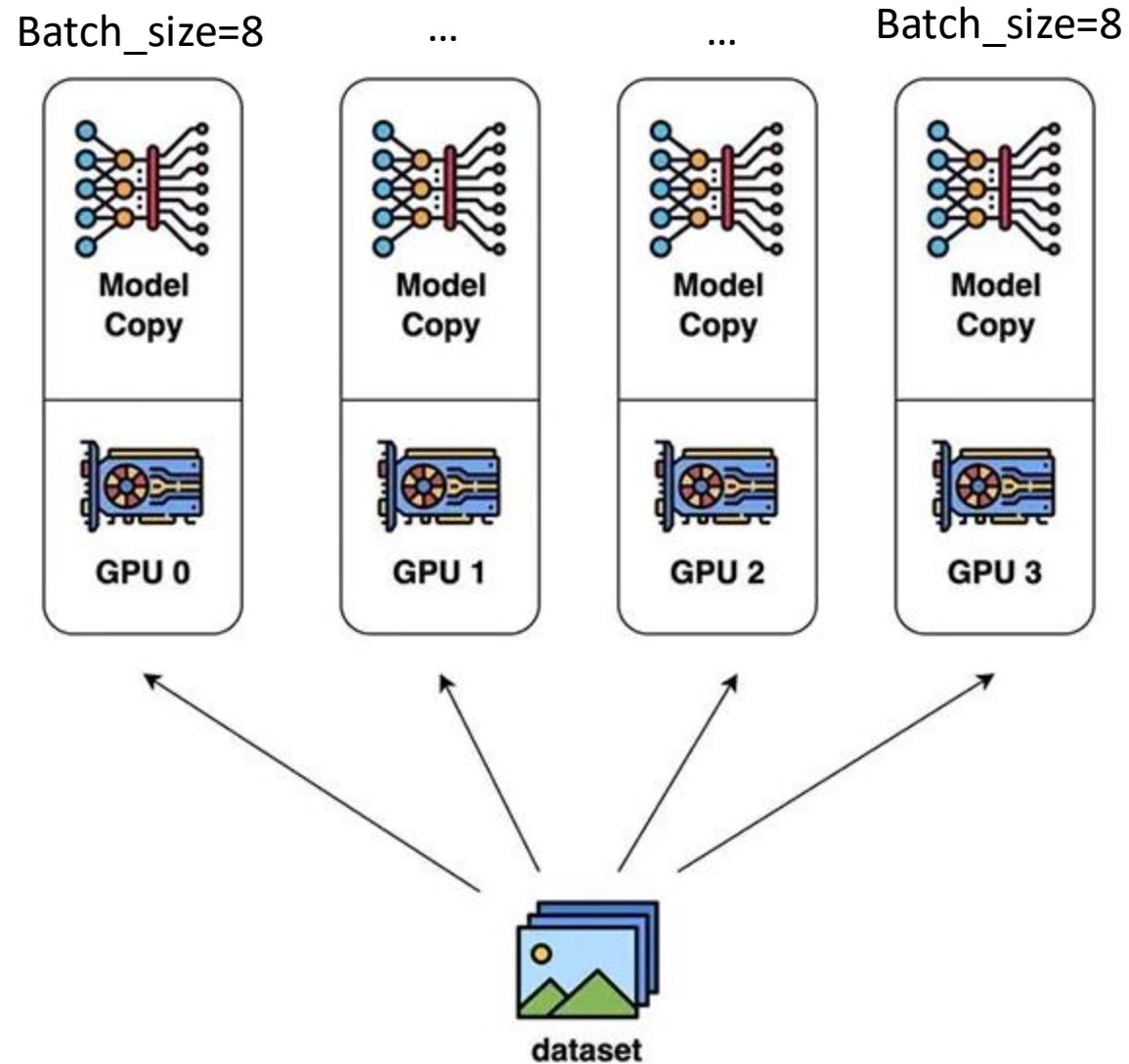
For $t = 1$ to T (iters)

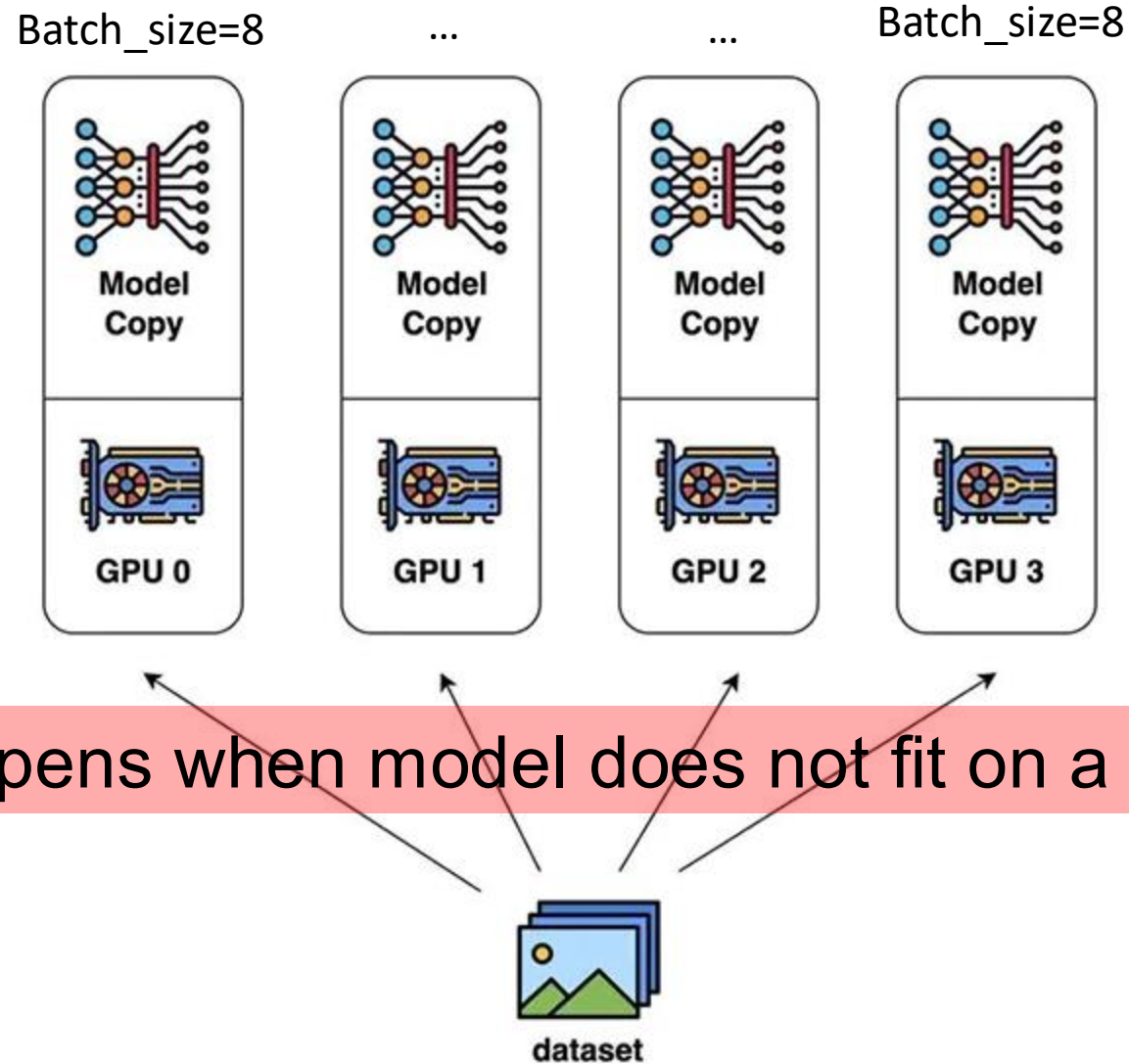
$$\Delta w = \eta \times \frac{1}{N} \sum_{i=1}^N \nabla \left(\text{loss}(f_w(x_i, y_i)) \right) \quad // \text{ compute derivative and update}$$

$w -= \Delta w$ // apply update

End

Data Parallelism (DP)





1985

LEARNING IN PARALLEL NETWORKS

BY GEOFFREY E. HINTON

Simulating learning in a probabilistic system

2012



Reflections of DL parallelization in early DL papers

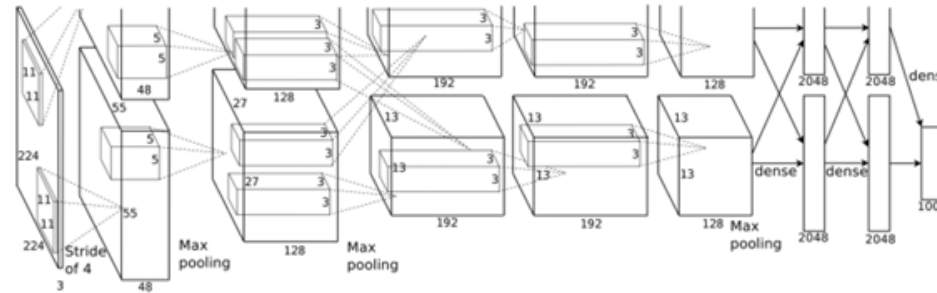


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Figure from AlexNet
[Krizhevsky et al., NeurIPS 2012],
[Krizhevsky et al., preprint, 2014]

2012

Reflections of DL parallelization in early DL papers

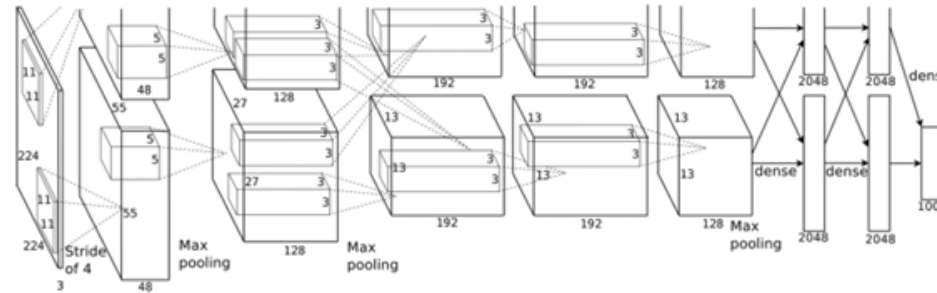


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Figure from AlexNet
[Krizhevsky et al., NeurIPS 2012],
[Krizhevsky et al., preprint, 2014]

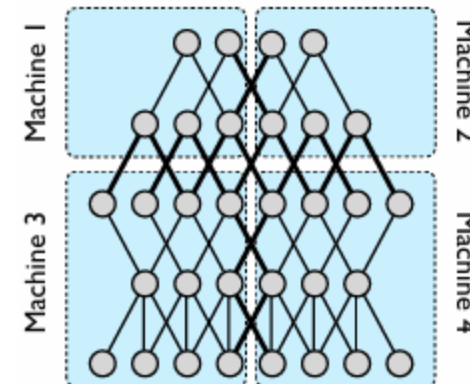
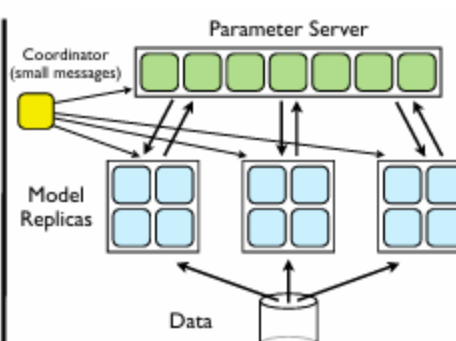
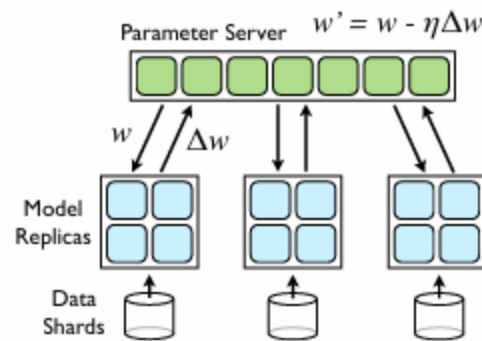
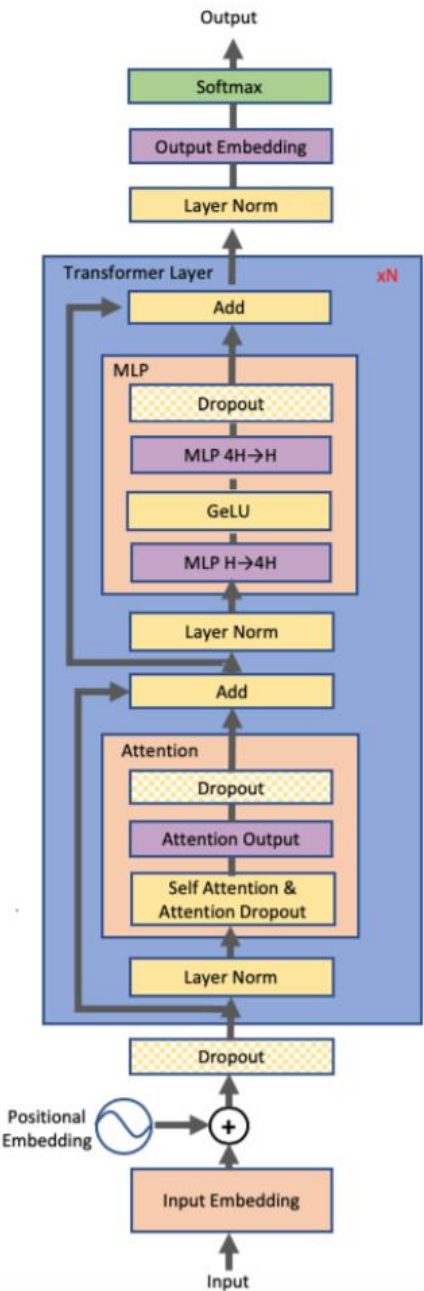


Figure from DistBelief
[Dean et al., NeurIPS 2012]

Large Model Training Challenges



	Bert-Large	GPT-2	Turing 17.2 NLG	GPT-3
Parameters	0.32B	1.5B	17.2B	175B
Layers	24	48	78	96
Hidden Dimension	1024	1600	4256	12288
Relative Computation	1x	4.7x	54x	547x
Memory Footprint	5.12GB	24GB	275GB	2800GB



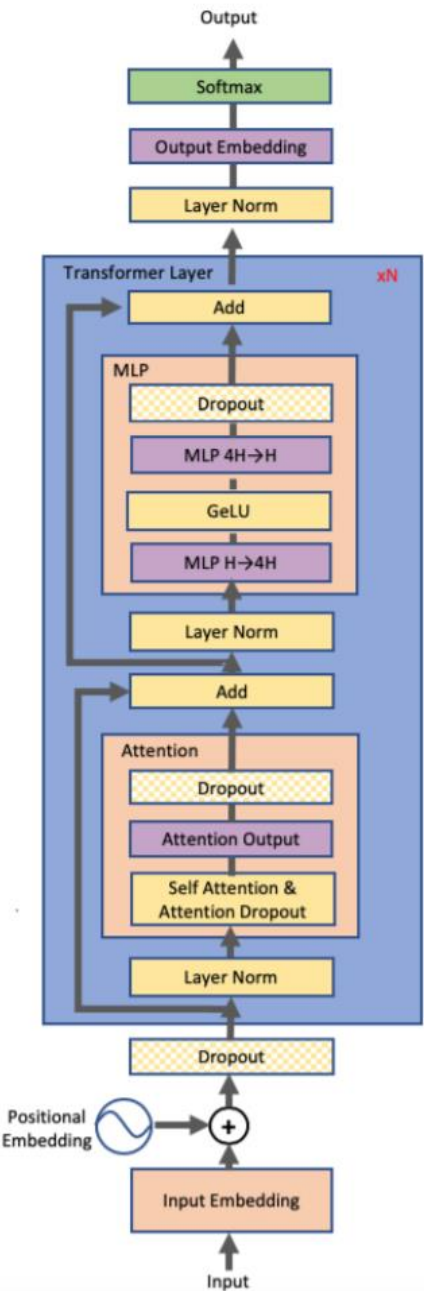
Large Model Training Challenges



	Bert-Large	GPT-2	Turing 17.2 NLG	GPT-3
Parameters	0.32B	1.5B	17.2B	175B
Layers	24	48	78	96
Hidden Dimension	1024	1600	4256	12288
Relative Computation	1x	4.7x	54x	547x
Memory Footprint	5.12GB	24GB	275GB	2800GB

Out of Memory

NVIDIA V100 GPU memory capacity: 16G/32G
NVIDIA A100 GPU memory capacity: 40G/80G



Modern Distributed Training

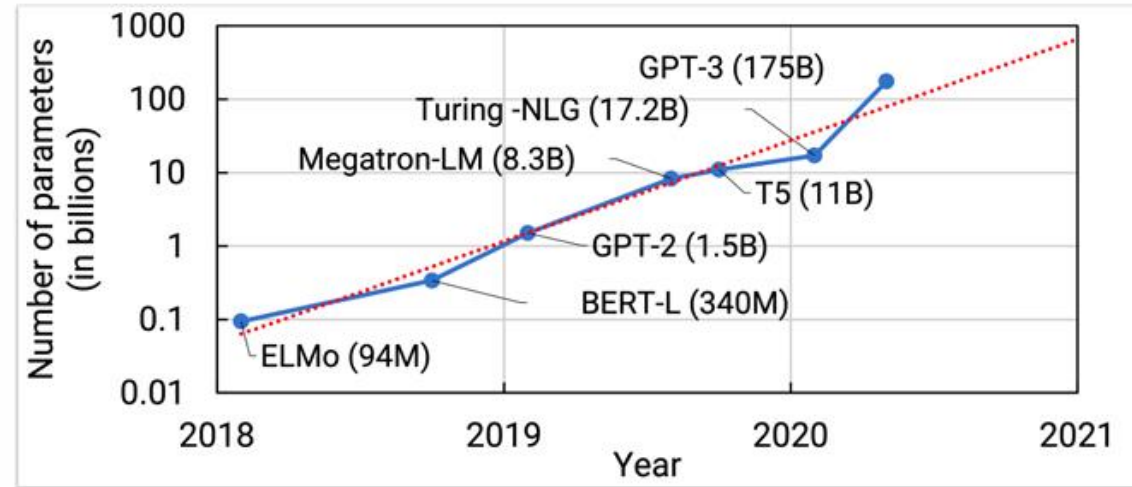
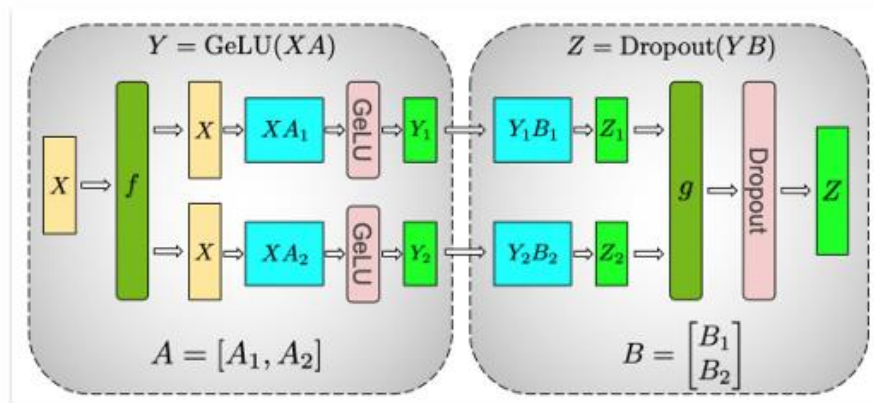
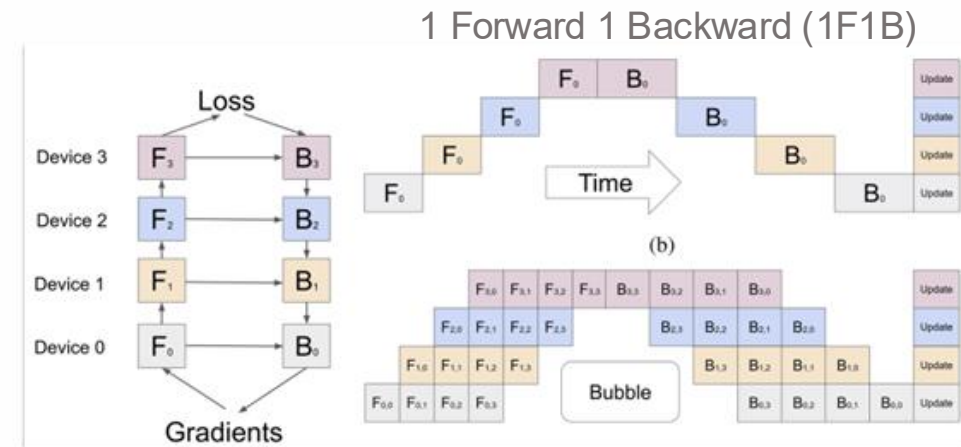


Figure from Nvidia



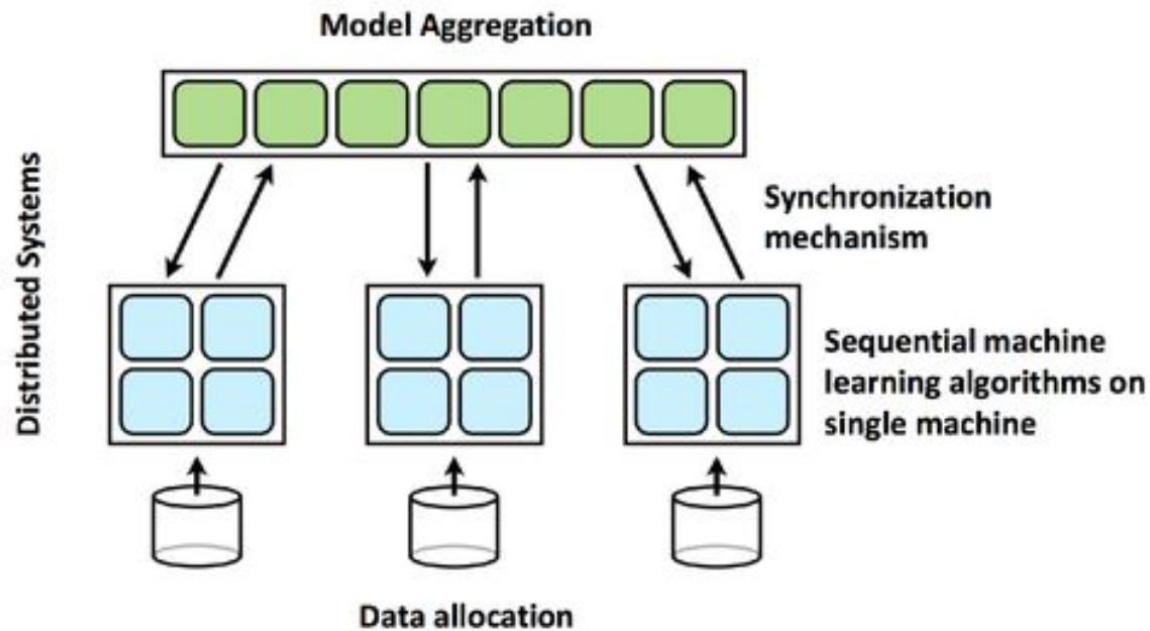
Matmul partitioning
[Shoeybi et al., ICML 2020]



Pipeline parallelism
[Huang et al., NeurIPS 2019]

- **Distributed Training**
 - Problem
 - Challenge
 - History
 - **Data Parallelism with Parameter Servers**

- (Centralized) Parameter Server
- (Decentralized) AllReduce
- Key assumption:
 - The model can fit into a (GPU) worker memory so we can create many model replicas



1. Partition the training data
2. Parallel training on different machines
3. Synchronize the local updates
4. Refresh local model with new parameters, then go to 2

2012

2016

```
import torch.nn.parallel as dist
from torch.nn.parallel import DistributedDataParallel as DDP

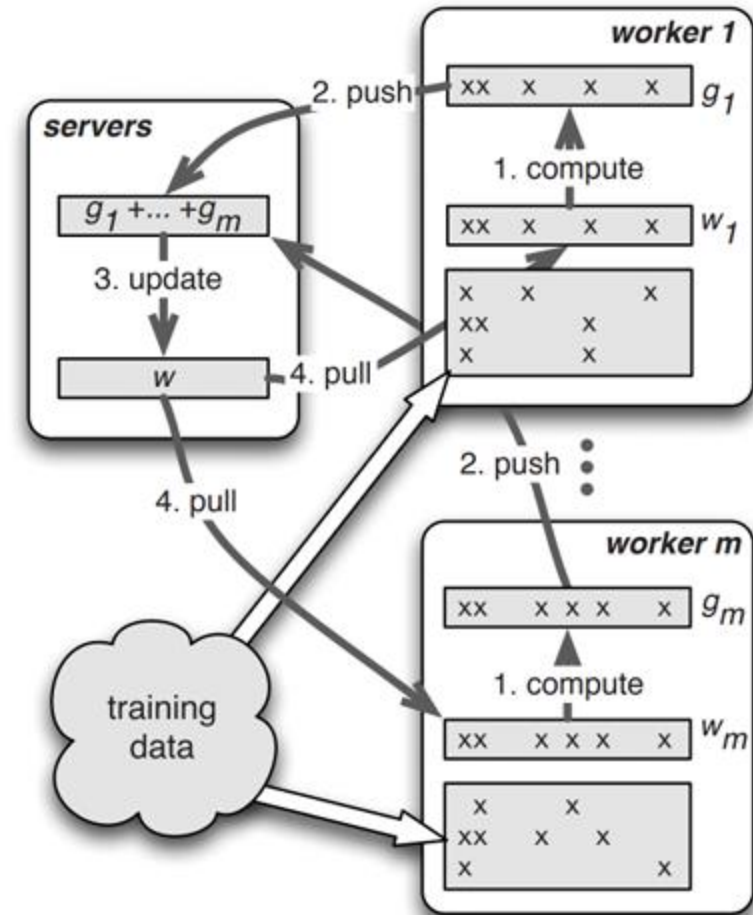
dist.init_process_group("nccl", rank=rank, world_size=world_size)
ddp_model = DDP(Model(), device_ids=[rank])

for batch in data_loader:
    loss = train_step(ddp_model, batch)
```

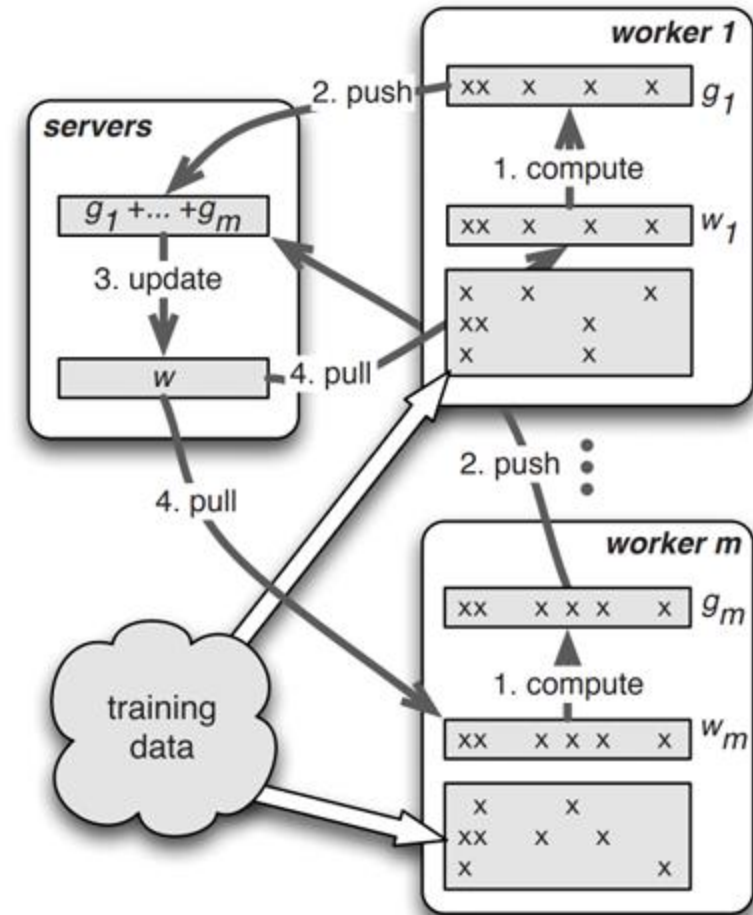
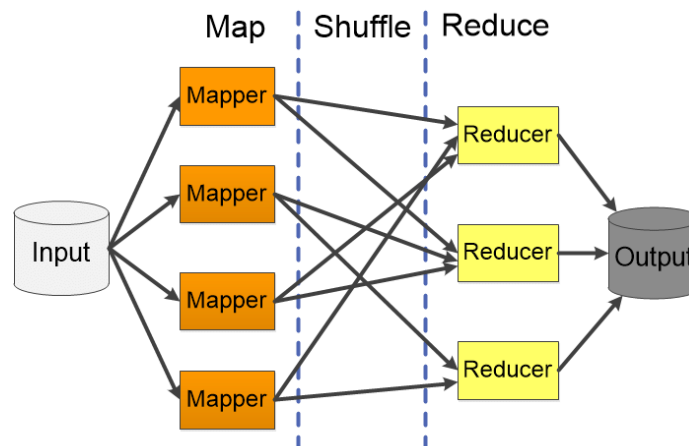
Sergeev et al., "Horovod: fast and easy distributed deep learning in TensorFlow". Preprint 2018.

Li et al., "PyTorch Distributed: Experiences on Accelerating Data Parallel Training". VLDB 2020.

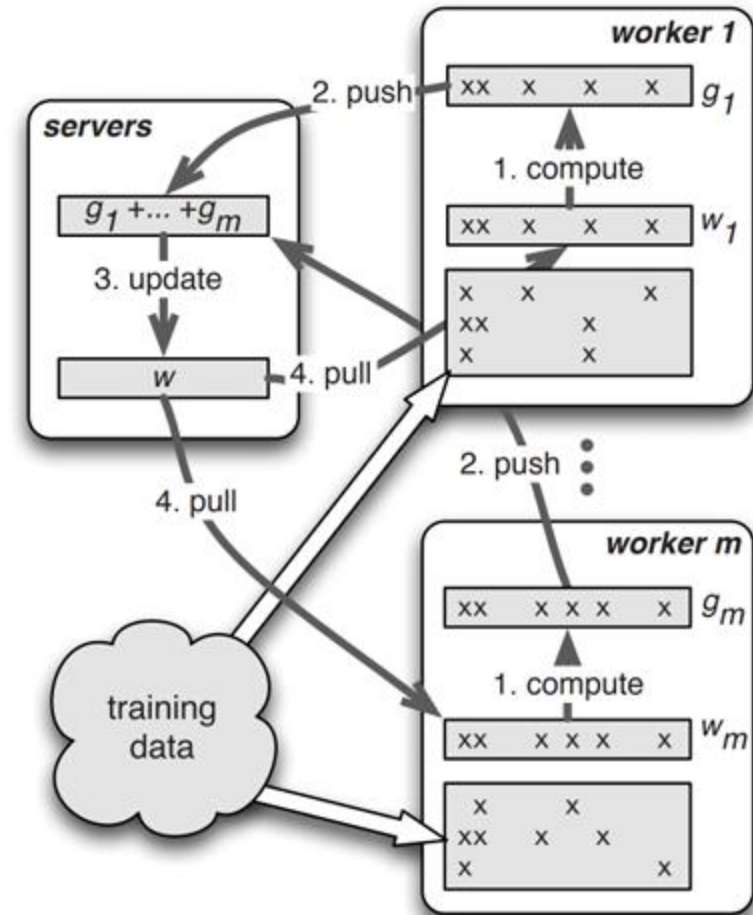
- Client (workers):
 - Pull(): a new iteration begins
 - Compute()
 - Push(): an iteration finishes
- Server (aggregator):
 - Update(): aggregate model updates



- Client (workers):
 - Pull(): a new iteration begins
 - Compute()
 - Push(): an iteration finishes
- Server (aggregator):
 - Update(): aggregate model updates
- Very similar to the spirit of MapReduce



- Client (workers):
 - Pull(): a new iteration begins
 - Compute()
 - Push(): an iteration finishes
- Server (aggregator):
 - Update(): aggregate model updates
- Very similar to the spirit of MapReduce
- Flexible and can be used in various ML algorithms, not just DL

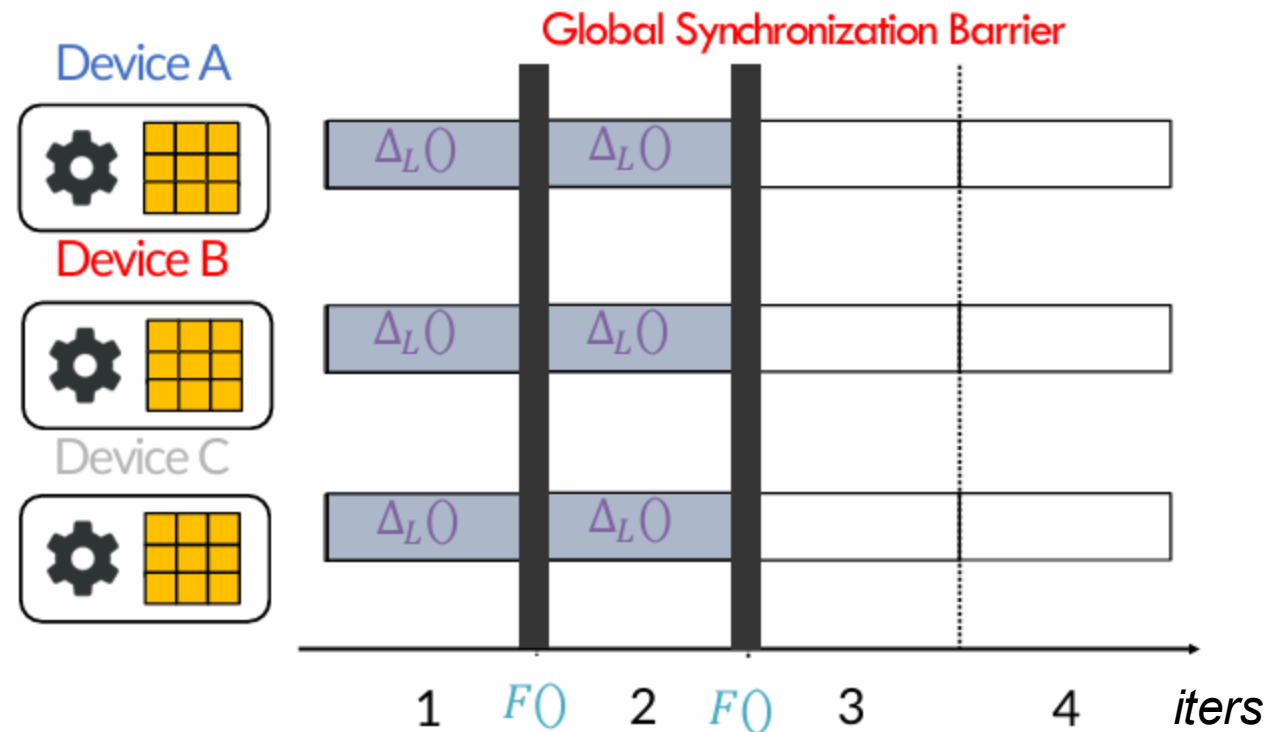


How to ensure correctness?

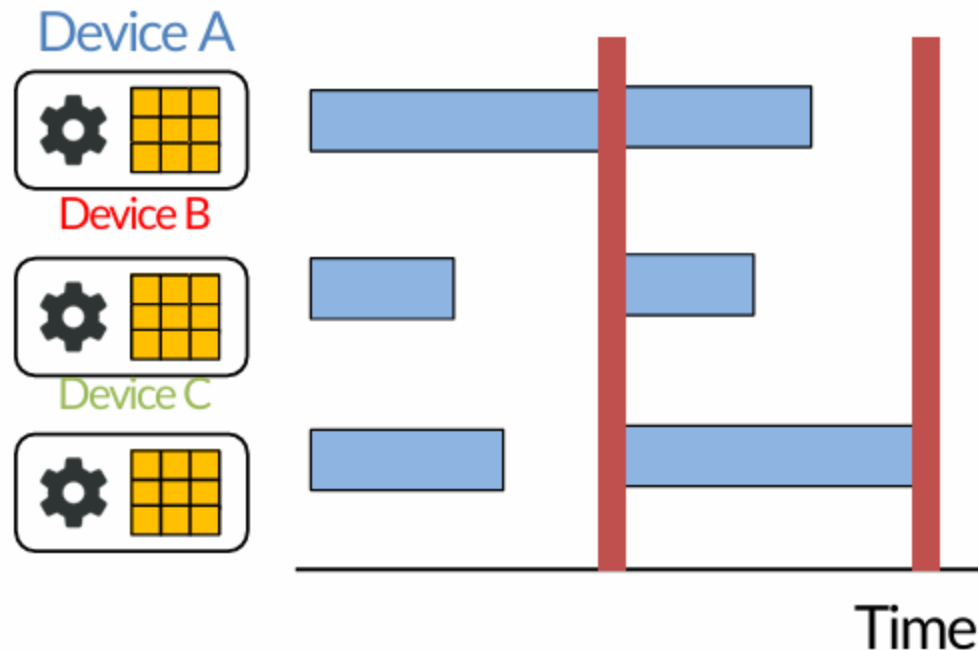
Bulk Synchronous Parallel: Consistency



$$\theta^{(t+1)} = \theta^{(t)} + \varepsilon \sum_{p=1}^P \nabla_{\mathcal{L}}(\theta^{(t)}, D_p^{(t)})$$



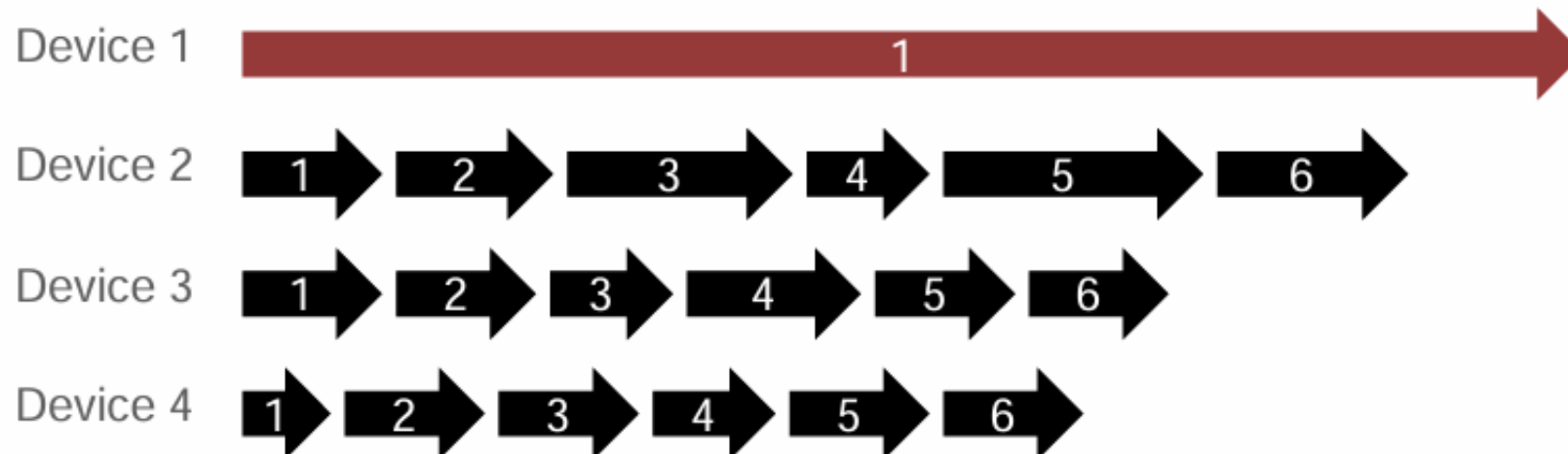
- **BSP suffers from stragglers**
 - Slow devices (stragglers) force all devices to wait
 - More devices → higher chance of having a straggler



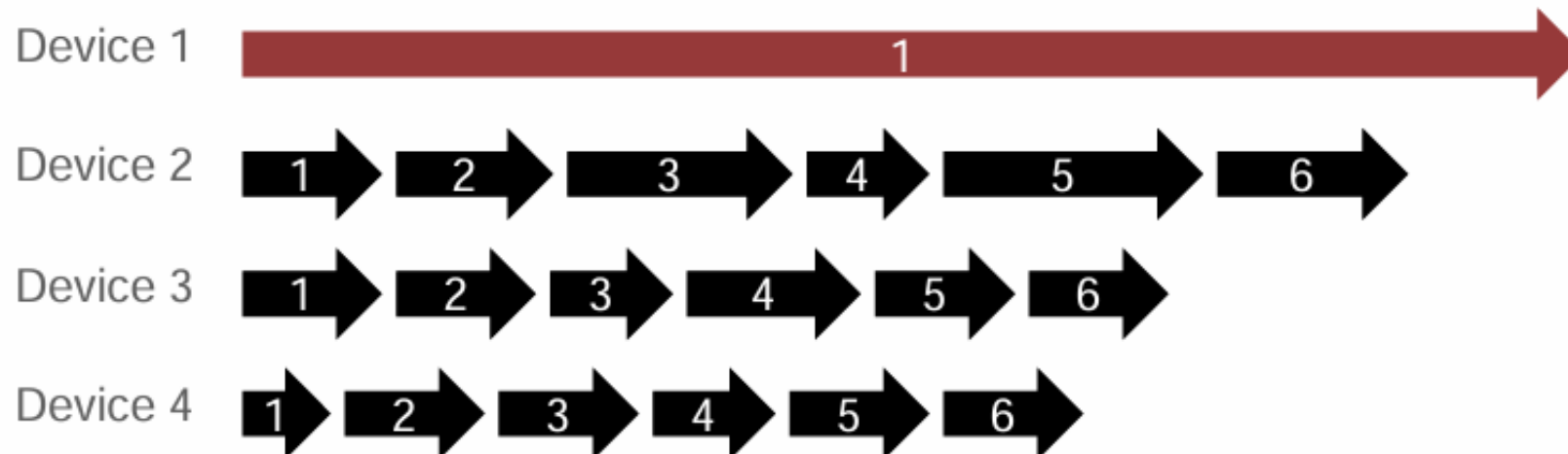
Asynchronous Communication (No Consistency)



- **Asynchronous (Async):** removes all communication barriers
 - Maximizes computing time
 - Transient stragglers will cause messages to be **extremely stale**
 - Ex: Device 2 is at $t = 6$, but Device 1 has only sent message for $t = 1$



- **Asynchronous (Async):** removes all communication barriers
 - Maximizes computing time
 - Transient stragglers will cause messages to be **extremely stale**
 - Ex: Device 2 is at $t = 6$, but Device 1 has only sent message for $t = 1$
- **Some Async software:** messages can be applied while computing $F()$, $\Delta_L()$
 - **Unpredictable behavior, can hurt statistical efficiency!**



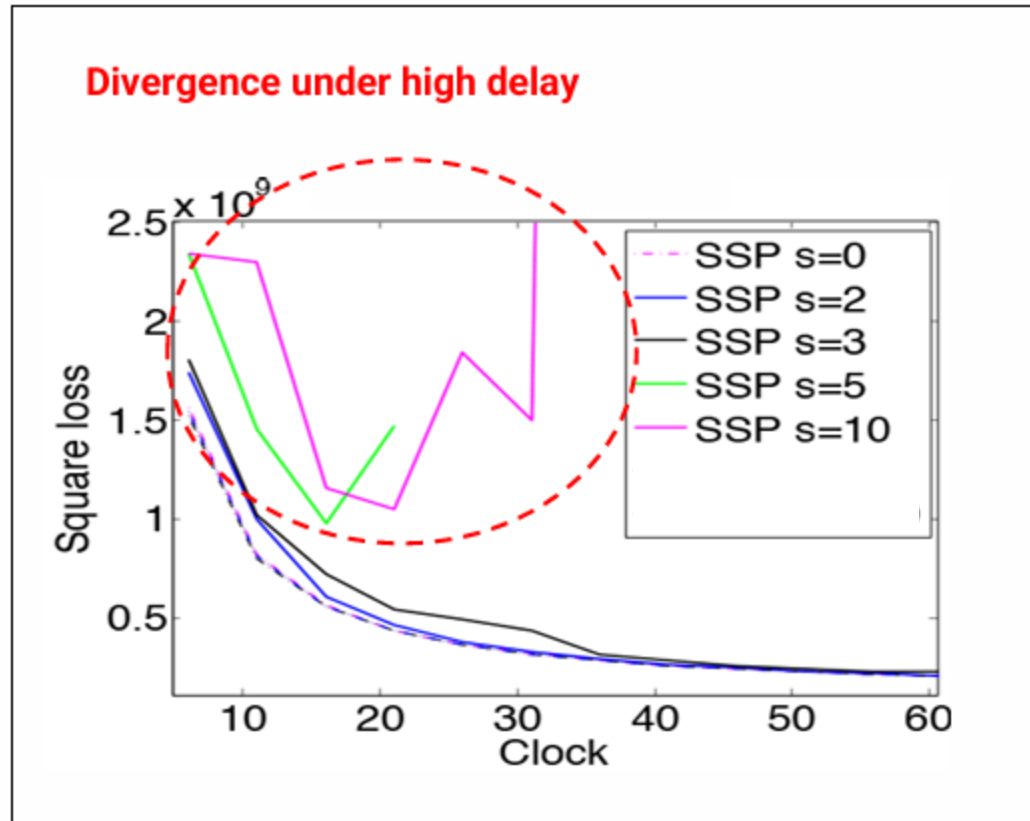
Bounded consistency models: Middle ground between BSP and fully-asynchronous (no-barrier)

e.g. Stale Synchronous Parallel (SSP): Devices allowed to iterate at different speeds

- Fastest & slowest device must not drift $> s$ iterations apart (in this example, $s = 3$)
- s is the **maximum staleness**



[Ho et al., 2013; Dai et al., 2015; Wei et al., 2015]



Discussions: How to control the staleness?

- Too small -> poor sys efficiency
- Too large -> poor stats efficiency

Data Parallelism with AllReduce



2012

2016

← Fast connections: NVLink, NVSwitch

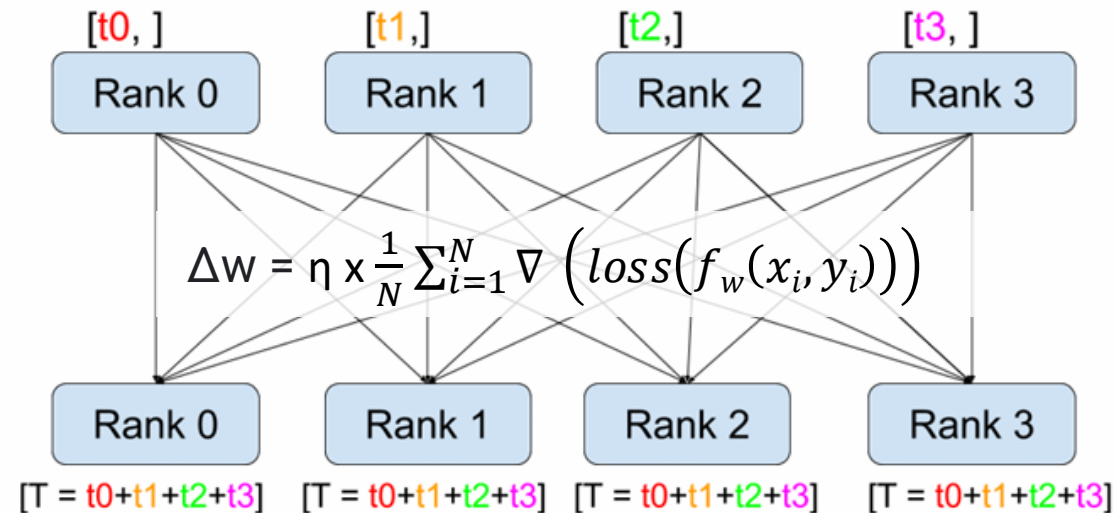
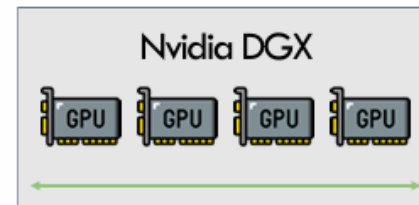


Figure from PyTorch Tutorials

- **Distributed Training**

- **Problem**
- **Challenge**
- **History**
- **Data Parallelism: The model can fit into a (GPU) worker memory**
 - (Centralized) Parameter Server
 - (Decentralized) AllReduce



DeepSeek-V3 Technical Report

DeepSeek-AI

`research@deepseek.com`

Introduce how DS team trains the 671B MoE model using 2.788M H800 GPU hours (\$5.6M)

- Multi-head attention
- Auxiliary-Loss-Free Load Balancing
- FP8 training